

NORTHPEAK OUTFITTERS · OBJECT-ORIENTED JAVA

Classes, Objects, Encapsulation, and Inheritance

How to invent your own type, protect its data, and let a related type share it — everything from here to a working object hierarchy, in one sitting.

Object-oriented Java is organized around four ideas — today covers two

Every one of these words will keep coming up for the rest of the course. Today you learn what two of them actually mean, in working code.

ENCAPSULATION Wrapping data and the rules that guard it into one unit, so nothing outside the class can put it in a state that makes no sense. **Today.**

INHERITANCE One class sharing its fields and methods with another, without copying a single line. **Today.**

ABSTRACTION Hiding how something works behind a simple contract. Builds directly on inheritance — coming in a later session.

POLYMORPHISM One line of code, several possible method bodies, chosen while the program is running. Also later.

Before either of today's two pillars means anything, you need the thing they both act on: a class you wrote yourself, with fields, a constructor, and methods. That is where this session actually starts.

Say the two words out loud: encapsulation is what an object refuses. Inheritance is what one class hands to another.

THE PROBLEM

Parallel arrays are three variables pretending to be one thing

No class yet — just three arrays, tracked by hand, and nothing stopping them from drifting out of sync.

RUNNABLE JAVA PROGRAM

```
public class ParallelShelf {
    public static void main(String[] args) {
        String[] tracking = {"NP-1001", "NP-1002"};
        double[] weights = {8.0, 25.0};
        String[] destinations = {"Denver", "Boise"};

        System.out.println(tracking[1] + " -> " + destinations[1]
            + " (" + weights[1] + " lbs)");
    }
}
```

Nothing in Java knows these three arrays are related. Nothing stops one from being longer than the others, nothing stops a weight of -40, and the fact that slot 1 means one package is a convention living only in your head.

A package is one thing. It should be one variable.

A class is a type you invent, and Java treats it like any other

You already use types you did not write. `String` and `Scanner` are classes someone else declared.

The class

The declaration. It says what every parcel *has* and what every parcel can *do*. Written once. No data in it — it is the form, not a filled-in form.

`new Parcel("NP-1001", 8.0, "Denver")` creates one object and hands you a reference to it — the same reference rule that applies to arrays.

The object

One actual parcel, made with `new`. It has its own values in its own memory. A thousand objects, one class.

Class is the blueprint. Object is the building. `new` is the construction.

The new keyword does three things, in one exact order

Every object you will ever create in Java goes through these three steps, whether you notice them or not.

- 1. ALLOCATE** Java reserves memory on the heap large enough for one Parcel's fields — nothing has a value yet.
- 2. CONSTRUCT** The constructor runs, using that memory, and gives every field a real value.
- 3. RETURN** new hands back a reference to that memory, which you store in a variable.

ONE LINE, THREE STEPS

```
Parcel p = new Parcel("NP-1001", 8.0, "Denver");
```

If you have ever wondered why a constructor "runs automatically" — this is the mechanism. new calls it for you, every time.

A type you invent owes three things

These three words are the spine of everything up to inheritance. Every slide from here answers one of them.

STATE	What the object knows — its fields. A parcel knows its tracking number, its weight and its destination.
BEHAVIOUR	What the object does — its methods. A parcel can price itself and print its own label, because it already has the data those answers need.
RULES	What the object refuses — encapsulation. A parcel will not accept a weight of -40, because no such parcel exists.

Most beginner classes stop after state. The rules are what make a type worth having.

SECTION ONE

Classes & Objects

Fields, constructors, and the guarantee that an object is never born half-built. The model for this section is STATE: what an object knows.

FIELDS

Fields are the variables every object of the type carries

Declared inside the class but outside every method. Each object gets its own copy of all of them.

PARCEL.JAVA &MDASH; STATE ONLY, FOR NOW

```
public class Parcel {  
    String tracking;  
    double weight;  
    String destination;  
}
```

That is a complete, compilable Java class. It has no behaviour and no rules yet, but it already does something the three parallel arrays could not: it says these three values belong to the same thing.

One file per public class, and the file name must match the class name exactly — `Parcel.java`.

DEFAULTS

Fields get default values; local variables do not

Same defaults as array slots, for the same reason — and the `null` is the one that will bite you.

RUNNABLE JAVA PROGRAM

```
public class NewParcel {  
    public static void main(String[] args) {  
        Parcel first = new Parcel();  
  
        System.out.println("tracking: " + first.tracking);  
        System.out.println("weight: " + first.weight);  
        System.out.println("destination: " + first.destination);  
    }  
}
```

OUTPUT

```
tracking: null  
weight: 0.0  
destination: null
```

This object is legal and useless. It weighs nothing and is going nowhere. The constructor exists to make this impossible.

CONSTRUCTORS

A constructor is the only chance to guarantee a valid object

It runs exactly once, at new, before anyone else can touch the object. Nothing else in Java has that position.

RUNNABLE JAVA PROGRAM

```
public class Parcel {  
    String tracking;  
    double weight;  
    String destination;  
  
    public Parcel(String tracking, double weight, String destination) {  
        this.tracking = tracking;  
        this.weight = weight;  
        this.destination = destination;  
    }  
}
```

NO RETURN TYPE not even void — that is what makes it a constructor and not a method

SAME NAME AS THE CLASS spelled identically, capital included

RUNS AT NEW `new Parcel("NP-1001", 8.0, "Denver")`

After this, `new Parcel()` with no arguments no longer compiles — and that is the point.

THE THIS KEYWORD

this distinguishes the field from the parameter with the same name

this means "the object this method was called on." Inside the constructor, that is the object being built.

Without this — compiles, does nothing

```
public Parcel(String tracking, double weight) {  
    tracking = tracking;  
    weight = weight;  
}
```

With this — assigns the parameter into the field

```
public Parcel(String tracking, double weight) {  
    this.tracking = tracking;  
    this.weight = weight;  
}
```

On the left, the nearer name wins: `tracking` refers to the parameter on both sides, so the parameter is assigned to itself and the field keeps its default. No error, no warning, and the object comes out empty.

Naming the parameter after the field is deliberate and standard. `this` is what makes it safe.

Write the constructor

Three blanks. Each one assigns a parameter into the field it belongs to.

REQUIREMENTS

- Every field must be set from its matching parameter.
- The parameters are named the same as the fields on purpose.
- After construction, `label()` must print real values.

Expected output

NP-1001 -> Denver (8.0 lbs)

Printing `null -> null (0.0 lbs)` means a blank is missing its `this`.

COMPLETE THE PLAN

```
public class Parcel {  
    String tracking;  
    double weight;  
    String destination;  
  
    public Parcel(String tracking, double weight, String destination) {  
        __.tracking = tracking;  
        this.weight = __;  
        this.__ = destination;  
    }  
}
```

Left of the dot is the object; right of the equals is the parameter

Read every constructor line the same way and the pattern stops being something to remember.

RUNNABLE SOLUTION (BODY)

```
this.tracking = tracking;  
this.weight = weight;  
this.destination = destination;
```

THIS.TRACKING	the field on the object being built — the thing that must survive
= TRACKING	the parameter that was passed to new — gone the moment the constructor returns
EVERY FIELD	"every field must be set" — a field you forget silently keeps its default

Three lines, three fields. If your class grows a fourth field, this constructor must grow a fourth line.

SECTION TWO

Methods

Behaviour that lives with the data it needs, and stops being passed around. The model for this section is BEHAVIOUR: what an object does.

METHODS

A method on the class already has the data, so it needs no arguments

This is the real argument for objects: behaviour moves next to the state it operates on, and the parameter lists collapse.

Free function — the caller must supply everything, in the right order

```
public static double cost(double weight) {  
    if (weight < 1) return 3.99;  
    return 24.99;  
}
```

Method on Parcel — weight is already here

```
public double shippingCost() {  
    if (weight < 1) return 3.99;  
    return 24.99;  
}
```

The second version cannot be called with the wrong package's weight, because there is no weight to pass. That whole class of bug is now impossible rather than merely unlikely.

Bugs you cannot express are better than bugs you remember not to write.

A method either answers a question or performs an action

The return type tells the caller which. `void` means "I did something"; anything else means "here is your answer".

RUNNABLE JAVA PROGRAM

```
public class Parcel {  
    String tracking;  
    double weight;  
    String destination;  
  
    public double shippingCost() {  
        if (weight < 1) {  
            return 3.99;  
        }  
        if (weight < 5) {  
            return 6.99;  
        }  
        if (weight < 20) {  
            return 12.99;  
        }  
        return 24.99;  
    }  
  
    public boolean isOversize() {  
        return weight > 20;  
    }  
}
```

A method that returns a value should not also print. Let the caller decide what to do with the answer.

toString decides what your object looks like when printed

Every class inherits a `toString` that prints a hash. Overriding it is the cheapest debugging investment you will ever make.

ADD THIS INSIDE PARCEL

```
@Override
public String toString() {
    return "Parcel[" + tracking + ", " + weight
        + " lbs, " + destination + "];"
}
```

BEFORE, THEN AFTER

```
Parcel@1b6d3586
Parcel[NP-1001, 8.0 lbs, Denver]
```

`System.out.println(parcel)` calls `toString()` for you. So does string concatenation, and so does printing an array of your objects.

Without it you get a memory address. With it you get information.

OVERLOADING

Overloading is one name for several methods with different parameters

Java picks by the argument types at the call site. The return type is not part of the decision.

RUNNABLE JAVA PROGRAM

```
public class Overloads {  
    public static void main(String[] args) {  
        System.out.println(half(12.99));  
        System.out.println(half(12));  
    }  
  
    public static double half(double amount) {  
        return amount / 2;  
    }  
  
    public static int half(int amount) {  
        return amount / 2;  
    }  
}
```

OUTPUT

```
6.495  
6
```

Two methods differing *only* in return type will not compile: the call site gives Java nothing to choose with.

Add a label method

Two blanks. Decide what the method hands back, and how it hands it back.

REQUIREMENTS

- The method answers a question, so it is not `void`.
- It builds one line of text from all three fields.
- It takes no parameters, because the object already has the data.

Expected output

NP-1001 -> Denver (8.0 lbs)

If your IDE says "missing return statement", the second blank is the one it is complaining about.

COMPLETE THE PLAN

```
public class Parcel {  
    String tracking;  
    double weight;  
    String destination;  
  
    public ____ label() {  
        ____ tracking + " -> " + destination  
            + " (" + weight + " lbs)";  
    }  
}
```

The return type is a promise, and return is how you keep it

Declaring `String` commits every path through the method to producing one.

RUNNABLE SOLUTION

```
public String label() {  
    return tracking + " -> " + destination  
        + " (" + weight + " lbs)";  
}
```

STRING	"not void" — the caller gets text back and decides whether to print it
RETURN	"builds one line of text" — the value leaves the method here
NO PARAMETERS	"the object already has the data" — <code>tracking</code> is a field, in scope everywhere in the class

`label()` returning text rather than printing it is what lets you put it in a file later without changing the method.

KNOWLEDGE CHECK

The constructor runs and the field is still empty

This compiles cleanly. Work out which tracking each side of the assignment refers to.

THINK IT THROUGH FIRST

WHAT DOES THIS PRINT?

```
public class Parcel {  
    String tracking;  
  
    public Parcel(String tracking) {  
        tracking = tracking;  
    }  
}  
  
Parcel p = new Parcel("NP-1001");  
System.out.println(p.tracking);
```

A. NP-1001

B. null

C. An empty line

D. It does not compile

The parameter was assigned to itself, so the field never changed

The nearer declaration wins. Inside the constructor, the bare name `tracking` is the parameter on both sides.

ANSWER

WHAT DOES THIS PRINT?

```
public class Parcel {
    String tracking;

    public Parcel(String tracking) {
        tracking = tracking;
    }
}

Parcel p = new Parcel("NP-1001");
System.out.println(p.tracking);
```

A. NP-1001

B. null

C. An empty line

D. It does not compile

WHY

The field keeps its default, and for a `String` that is `null` — which prints as the four characters `null`, not as a blank line. `this.tracking = tracking;` fixes it.

SECTION THREE

Encapsulation

The part that turns a bag of fields into a type worth trusting. The model for this section is RULES: what an object refuses.

Public fields mean anyone can put your object into a state that cannot exist

Nothing about the class we have written so far prevents this, and nothing reports it.

BROKEN PROGRAM

```
public class OpenDoor {  
    public static void main(String[] args) {  
        Parcel broken = new Parcel("NP-9999", 8.0, "Denver");  
  
        broken.weight = -40.0;  
  
        System.out.println(broken.label());  
        System.out.println("cost: " + broken.shippingCost());  
    }  
}
```

OUTPUT

```
NP-9999 -> Denver (-40.0 lbs)  
cost: 3.99
```

A parcel weighing -40 lbs is being billed at the cheapest tier. No exception, no warning — just a wrong invoice.

private closes the door; a getter and setter open a controlled window

The field becomes unreachable from outside the class. Every read and every write now goes through a method you wrote. This is data hiding.

RUNNABLE JAVA PROGRAM

```
public class Parcel {  
    private String tracking;  
    private double weight;  
  
    public double getWeight() {  
        return weight;  
    }  
  
    public boolean setWeight(double newWeight) {  
        if (newWeight <= 0 || newWeight > 150) {  
            System.out.println("REJECTED weight " + newWeight  
                               + " for " + tracking);  
            return false;  
        }  
        this.weight = newWeight;  
        return true;  
    }  
}
```

The naming convention is not optional: a getter for weight is called `getWeight`, and its setter is called `setWeight`. Every Java tool, from your IDE to frameworks you have not met yet, relies on that convention.

A getter that only returns is not the point. The setter that can refuse is.

THE POINT

The whole value of a setter is that it can say no

Returning `boolean` lets the caller find out.

RUNNABLE JAVA PROGRAM

```
public class SetterSaysNo {  
    public static void main(String[] args) {  
        Parcel good = new Parcel("NP-1001", 8.0, "Denver");  
        System.out.println(good);  
  
        boolean accepted = good.setWeight(-40.0);  
        System.out.println("accepted: " + accepted);  
        System.out.println("still: " + good.getWeight() + " lbs");  
    }  
}
```

OUTPUT

```
Parcel[NP-1001, 8.0 lbs, Denver]  
REJECTED weight -40.0 for NP-1001  
accepted: false  
still: 8.0 lbs
```

The object defended itself. Every caller, everywhere, in code not yet written, gets that protection for free.

Not every field deserves a setter

Generating a getter and setter for every field is a habit, not a design. Ask what is actually allowed to change.

FIELD	GETTER	SETTER	WHY
tracking	yes	no	Assigned once at creation. A parcel whose tracking number changes is a different parcel
weight	yes	yes, guarded	Re-weighing happens; a negative weight does not
destination	yes	yes, guarded	Addresses get corrected; blank ones are never correct
created	yes, static	no	The class maintains it. Letting anyone set it makes the number meaningless

Mark a field `final` and the compiler enforces "assigned once, in the constructor, never again" — `private final String tracking;`

Fewer setters means fewer ways for the object to become wrong.

If the constructor does not validate, the door was open at birth

A guarded setter is worthless if new can bypass it. Have the constructor call the setter.

ONE RULE, ONE PLACE

```
public Parcel(String tracking, double weight, String destination) {  
    this.tracking = tracking;  
    this.weight = 0.0;  
    this.destination = "UNSET";  
    setWeight(weight);  
    setDestination(destination);  
    created++;  
}
```

```
NEW PARCEL("NP-9999", -40.0, "BOISE")
```

```
REJECTED weight -40.0 for NP-9999  
Parcel[NP-9999, 0.0 lbs, Boise]
```

⚠ This is still not perfect, and you should notice why

The bad weight was refused, but the object was created anyway — at 0.0 lbs, which prices at the cheapest tier. A constructor cannot return "false". A later lesson fixes exactly this by letting the constructor throw, so the broken parcel never exists at all.

Write the rule once, in the setter, and call it from the constructor. Never copy the if.

State, behaviour, rules

State

private fields — what the object knows, and nobody outside can reach

Behaviour

public methods — what it does, using data it already holds

Rules

guarded setters and a validating constructor — what it refuses, on behalf of every caller who forgets

A class with only state is a struct. The rules are what make it a type. Next: sharing all of this with a second, related class.

Guard the setter

Three blanks. The rule is stated in the requirements; your job is to write it so that no caller can get around it.

REQUIREMENTS

- A weight of zero or less is never valid.
- The caller must be told the write was refused.
- On a good value, the field is updated.

Expected output

REJECTED weight -40.0 for NP-1001

accepted: false

still: 8.0 lbs

If the weight becomes -40.0 anyway, the assignment is outside the guard instead of after it.

COMPLETE THE PLAN

```
public boolean setWeight(double newWeight) {  
    if (newWeight ____ 0 || newWeight > 150) {  
        System.out.println("REJECTED weight " + newWeight  
            + " for " + tracking);  
        return ____;  
    }  
    ____ .weight = newWeight;  
    return true;  
}
```

The guard returns early, so the assignment below it is unreachable for bad input

The check goes above the work it is protecting.

RUNNABLE SOLUTION (BODY)

```
if (newWeight <= 0 || newWeight > 150) {  
    System.out.println("REJECTED weight " + newWeight  
        + " for " + tracking);  
    return false;  
}  
this.weight = newWeight;  
return true;
```

<= 0 "zero or less is never valid" — zero is included, so < alone is wrong

RETURN FALSE "the caller must be told" — and it exits before the assignment

THIS.WEIGHT "on a good value the field is updated" — the parameter is newWeight, so this is not strictly required here, but it says which one you meant

Guard, return, then do the work. The good path ends up un-nested and easy to read.

Access modifiers decide who is allowed to reach a member

Start everything `private` and open only what a caller genuinely needs.

MODIFIER	REACHABLE FROM	USE IT FOR
<code>private</code>	Inside this class only	Every field, and any method that is an implementation detail
<code>public</code>	Anywhere	The methods that are the point of the class
<code>protected</code>	This class, subclasses, same package	Members a subclass needs — see Section Four, coming up
(none)	Same package only	The default. Fine within one package, surprising across them

"Private fields, public methods" is the default posture, not a rule to memorise for an exam.

STATIC

static belongs to the class; everything else belongs to the object

One copy shared by every instance, and it exists before any object does. That is why `main` is static.

RUNNABLE JAVA PROGRAM

```
public class Parcel {  
    private static int created = 0;  
  
    private final String tracking;  
    private double weight;  
  
    public Parcel(String tracking, double weight) {  
        this.tracking = tracking;  
        setWeight(weight);  
        created++;  
    }  
  
    public static int getCreated() {  
        return created;  
    }  
}
```

`PARCEL.GETCREATED()` called on the *class* — no object needed

`PARCEL.GETWEIGHT()` called on an *object* — the answer differs per parcel

THE RULE a static method cannot touch instance fields, because it does not know which object you meant

Use static for counts across all instances, constants, and utilities. Not as a way to dodge writing a constructor.

An array of objects is what the parallel arrays should have been

One array, one type, and the pairing is now impossible to break because there is nothing to keep in step.

RUNNABLE JAVA PROGRAM

```
public class Manifest {  
    public static void main(String[] args) {  
        Parcel[] shelf = new Parcel[3];  
  
        shelf[0] = new Parcel("NP-1001", 8.0, "Denver");  
        shelf[1] = new Parcel("NP-1002", 25.0, "Boise");  
        shelf[2] = new Parcel("NP-1003", 0.5, "Reno");  
  
        for (int i = 0; i < shelf.length; i++) {  
            Parcel p = shelf[i];  
            System.out.println(p.label() + " $" + p.shippingCost());  
        }  
    }  
}
```

OUTPUT

```
NP-1001 -> Denver (8.0 lbs) $12.99  
NP-1002 -> Boise (25.0 lbs) $24.99  
NP-1003 -> Reno (0.5 lbs) $3.99
```

A sort can no longer desynchronise anything, because there is only one array.

KNOWLEDGE CHECK

One of these two variables is shared and one is not

Two objects are created. Work out what each object ends up holding.

THINK IT THROUGH FIRST

WHAT DOES THIS PRINT?

```
public class Counter {  
    static int made = 0;  
    int id;  
  
    public Counter() {  
        made++;  
        id = made;  
    }  
  
    public static void main(String[] args) {  
        new Counter();  
        Counter third = new Counter();  
        System.out.println(third.id + " " + made);  
    }  
}
```

A. 1 1

B. 2 2

C. 2 1

D. 1 2

made is shared; id is a separate value in each object

ANSWER

WHAT DOES THIS PRINT?

```
public class Counter {
    static int made = 0;
    int id;

    public Counter() {
        made++;
        id = made;
    }

    public static void main(String[] args) {
        new Counter();
        Counter third = new Counter();
        System.out.println(third.id + " " + made);
    }
}
```

A. 1 1

B. 2 2

C. 2 1

D. 1 2

WHY

made is static, so both constructor calls incremented the same variable — it ends at 2. id is an instance field, so third has its own copy holding 2. This is the auto-incrementing-ID pattern, and also why it breaks under concurrency.

SECTION FOUR

Inheritance

One class handing its fields and methods to another, without copying either.

THE PROBLEM

Two shipment types that are 90% the same class, twice

NorthPeak adds an express service. It differs from ground in one number. Here is what copy-paste costs you.

Ground.java

```
public class Ground {  
    private final String tracking;  
    private final double weight;  
    public double shippingCost() {  
        return groundRate();  
    }  
}
```

Express.java — identical except one line

```
public class Express {  
    private final String tracking;  
    private final double weight;  
    public double shippingCost() {  
        return groundRate() + 3.00;  
    }  
}
```

Change the tracking-number format and you edit two files. Add a third service and you edit three. And an array can hold one type or the other, never both.

Inheritance exists so the shared part is written once and the different part is written once.

Every shipment type ahead builds on one shared parent class

Before extends means anything, here is the exact class every shipment type in this section extends.

SHIPMENT.JAVA &MDASH; EVERY EXAMPLE AHEAD BUILDS ON THIS FILE

```
public class Shipment {
    private final String tracking;
    private final double weight;

    public Shipment(String tracking, double weight) {
        this.tracking = tracking;
        this.weight = weight;
    }

    public double getWeight() {
        return weight;
    }

    public double shippingCost() {
        return 0.0;
    }

    @Override
    public String toString() {
        return getClass().getSimpleName() + " " + tracking
            + " (" + weight + " lbs)";
    }
}
```

`shippingCost()` returns 0.0 on purpose — a bare shipment is never actually shipped. Every subclass overrides that one method, and `getClass().getSimpleName()` is why `toString()` prints "Ground" or "Express" without any subclass writing its own.

Every extends example from here on adds to this file. Nothing replaces it.

extends means "is a kind of", and shares without copying

The subclass gets every field and method of the parent, then adds or replaces only what differs. This is the IS-A relationship.

RUNNABLE JAVA PROGRAM

```
public class Ground extends Shipment {  
    public Ground(String tracking, double weight) {  
        super(tracking, weight);  
    }  
  
    @Override  
    public double shippingCost() {  
        double w = getWeight();  
        if (w < 1) {  
            return 3.99;  
        }  
        if (w < 5) {  
            return 6.99;  
        }  
        if (w < 20) {  
            return 12.99;  
        }  
        return 24.99;  
    }  
}
```

Ground declares no fields and no getters. It has `getWeight()` because `Shipment` has it. The only thing in this file is what makes ground shipping different.

Java allows one parent class per subclass. Say it out loud as a sentence: "a ground shipment IS A shipment."

SUPER

super(...) must be the first line, because the parent has to exist first

You cannot set up the specialised part of an object before the general part is built. Java enforces the order.

RUNNABLE JAVA PROGRAM

```
public class Freight extends Shipment {  
    private final int pallets;  
  
    public Freight(String tracking, double weight, int pallets) {  
        super(tracking, weight);  
        this.pallets = pallets;  
    }  
}
```

SUPER(...)

calls the parent constructor — must be the first statement, no exceptions

THEN THIS.PALLETS

the subclass's own field, set after the parent is complete

IF YOU OMIT IT

Java inserts `super ( )` for you, and the code fails to compile if the parent has no no-argument constructor

SUPER.FIELD / SUPER.METHOD()

outside a constructor, `super` also reaches the parent's version of a field or method you have overridden

A constructor error mentioning "no suitable constructor" almost always means a missing or wrong `super ( . . . )`.

WHAT IS INHERITED

A subclass inherits members, not constructors, and not private access

"Inherited" and "reachable" are different questions, and the difference is where protected earns its place.

MEMBER OF THE PARENT	INHERITED?	REACHABLE FROM THE SUBCLASS?
public method or field	Yes	Yes
protected method or field	Yes	Yes — this is what protected is for
private field	Yes, it exists in the object	No — use the parent's public getter
Constructor	No	Call it with <code>super (. . .)</code>
static member	Yes	Yes, but there is still only one copy, shared by the whole family

This is why `Ground.shippingCost()` calls `getWeight()` and not `weight`. The field is there in the object; the subclass just is not allowed to reach past the parent's rules to get at it.

Encapsulation still applies inside the family. That is a feature, not an inconvenience.

Overriding replaces an inherited method for one subclass

Same name, same parameters, compatible return type. Different body. `super.method()` calls the version you replaced.

RUNNABLE JAVA PROGRAM

```
public class Express extends Ground {
    public Express(String tracking, double weight) {
        super(tracking, weight);
    }

    @Override
    public double shippingCost() {
        return super.shippingCost() + 3.00;
    }

    @Override
    public String serviceName() {
        return "Express";
    }
}
```

OUTPUT

```
Ground NP-1001 (8.0 lbs) $12.99
Express NP-1002 (8.0 lbs) $15.99
```

`super.shippingCost()` reuses the entire ground rate ladder. Express owns one number and inherits the rest.

@Override is a compiler check, not a comment

It tells Java "I believe I am replacing an inherited method." If you are wrong, the build fails instead of the behaviour silently going missing.

Without @Override — a typo compiles as a brand new method

```
public class Express extends Ground {  
    public double shippingcost() {  
        return super.shippingCost() + 3.00;  
    }  
}
```

With @Override — the same typo will not compile

```
public class Express extends Ground {  
    @Override  
    public double shippingcost() {  
        return super.shippingCost() + 3.00;  
    }  
}
```

On the left, Express now has two methods: the inherited shippingCost and a new, never-called shippingcost. Every express shipment silently bills at the ground rate.

Write @Override every single time. It costs nothing and it catches the bug class that is hardest to see.

Overriding replaces; overloading adds

Two words one letter apart that do opposite things. The parameter list is how you tell them apart.

	OVERRIDING	OVERLOADING
Where	In a subclass	In the same class (or a subclass)
Signature	Must match the parent's exactly	Must differ in parameters
Chosen	At run time , by the object's real type	At compile time , by the argument types
Purpose	Change behaviour for one kind of thing	Offer the same idea for different inputs
Return type	The same type, or a subtype of it	Irrelevant — cannot distinguish overloads

Change the parameter list of a method you meant to override and you have silently written an overload instead. `@Override` catches it.

Add the freight tier

Four blanks. Freight is billed per pallet, not per pound, so it replaces the pricing entirely.

REQUIREMENTS

- Freight is a kind of shipment.
- It carries its own extra field: the pallet count.
- The parent must be constructed before the pallet count is stored.
- Freight costs \$75 base plus \$25 per pallet.

Expected output

Freight NP-1003 (900.0 lbs) \$125.0

"Constructor Shipment in class Shipment cannot be applied to given types" means the second blank is missing or has the wrong arguments.

COMPLETE THE PLAN

```
public class Freight ____ Shipment {
    private final int pallets;

    public Freight(String tracking, double weight, int pallets) {
        ____ (tracking, weight);
        this.pallets = pallets;
    }

    @____
    public double shippingCost() {
        ____ 75.00 + (pallets * 25.00);
    }
}
```

Every blank came from a sentence, and one of them is an ordering rule

The `super(...)` line is not style. Java will not compile the class without it in that position.

RUNNABLE SOLUTION

```
public class Freight extends Shipment {
    private final int pallets;

    public Freight(String tracking, double weight, int pallets) {
        super(tracking, weight);
        this.pallets = pallets;
    }

    @Override
    public double shippingCost() {
        return 75.00 + (pallets * 25.00);
    }
}
```

EXTENDS

"is a kind of shipment" — the IS-A relationship in one word

SUPER(...)

"the parent must be constructed first" — and it must be the first statement

@OVERRIDE

freight replaces the pricing rather than adding to it — there is no super call

Freight ignores weight entirely and still inherits it, because a shipment has a weight even when the price does not depend on it.

KNOWLEDGE CHECK

The variable says Parent and the object says Child

Which one decides which body of name () runs?

THINK IT THROUGH FIRST

WHAT DOES THIS PRINT?

```
public class Parent {  
    public String name() {  
        return "parent";  
    }  
}  
  
public class Child extends Parent {  
    @Override  
    public String name() {  
        return "child";  
    }  
}  
  
Parent p = new Child();  
System.out.println(p.name());
```

A. parent

B. child

C. It does not compile

D. It throws at run time

The object actually built is what runs, not the type of the variable

ANSWER**WHAT DOES THIS PRINT?**

```
public class Parent {  
    public String name() {  
        return "parent";  
    }  
}  
  
public class Child extends Parent {  
    @Override  
    public String name() {  
        return "child";  
    }  
}  
  
Parent p = new Child();  
System.out.println(p.name());
```

A. parent

B. child

C. It does not compile

D. It throws at run time

WHY

p was declared as `Parent`, which only decides that calling `name()` is legal. The object built with `new Child()` is a `Child`, and its overridden version is what actually runs. A method that only exists on `Child` would not compile through p, though — that boundary is the subject of a later lesson.

What you can defend after today

State

private fields, set once by a validating constructor

Behaviour

methods that live next to the data they use, overloaded by parameter list, never by return type

Rules

a guarded setter, called from the constructor, is what makes a class a type instead of a bag of fields

Inheritance

extends shares fields and methods without copying; `super(...)` builds the parent first; `@Override` replaces one behaviour on purpose

Two of the four pillars of object-oriented Java, done. Abstraction and polymorphism build directly on today's inheritance — that is next.